

GPU-Accelerated Physics-Based Urban Flood Modelling

A production comparison of CuPy/CUDA and JAX/XLA

Sanjeev Kumawat

M.Sc. Student, Indian Institute of Technology Hyderabad

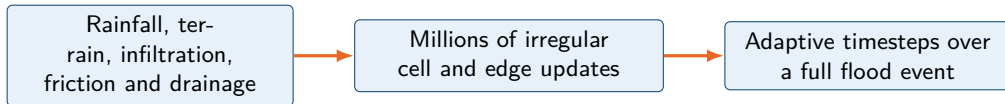
Under the guidance of **Prof. Udit Bhatia**

Associate Professor, Indian Institute of Gandhinagar



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

1. City-scale flood physics creates a computational bottleneck



Scientific need

Urban flooding requires conservation-based simulation of rapidly changing wet and dry flow over complex topography and coupled drainage infrastructure.

Computational need

Real-time or near-real-time use demands substantially faster execution without changing the equations, forcing, numerical controls or physical interpretation.

Acceleration matters only if numerical fidelity is preserved.

2. Research question: can acceleration preserve the model?

Central research question

Can the **same physics-based urban flood model** be implemented through two GPU programming approaches while preserving numerical equivalence, and how do their measured computational performances compare?

Objectives

- ▶ Implement equivalent GPU execution paths.
- ▶ Hold physics, inputs and numerical controls fixed.
- ▶ Test cross-backend parity and repeatability.
- ▶ Compare synchronized GPU runtimes.

Evidence required

1. Scientific model and discretization
2. Controlled implementation correspondence
3. Predefined correctness gates
4. Performance only after parity passes

Numerical verification is the prerequisite; observed-flood validation is a separate question.

3. The scientific core is the two-dimensional shallow-water model

Conservative balance law

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}}{\partial x} + \frac{\partial \mathbf{G}}{\partial y} = \mathbf{S}, \quad \mathbf{U} = [h, hu, hv]^T.$$

What the state represents

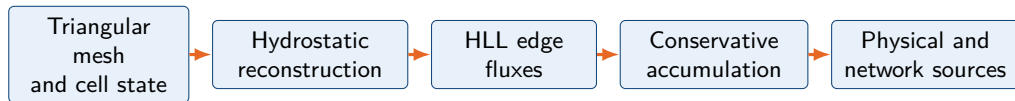
- ▶ h : water depth
- ▶ hu, hv : depth-integrated momentum
- ▶ Fluxes transport conserved water and momentum
- ▶ Sources represent terrain and coupled processes

Why finite volume?

The method balances fluxes across cell edges, directly supports conservation on an unstructured triangular mesh, and accommodates irregular urban geometry, wet-dry fronts, terrain steps and boundary interfaces.

The physics defines what must be preserved by both GPU paths.

4. The flood model is computed as a staged conservative update



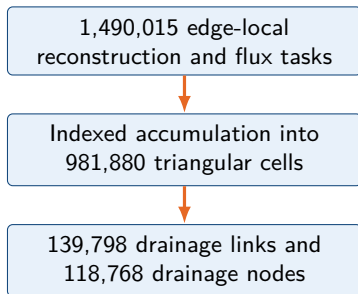
$$\mathbf{U}_i^{n+1} = \mathbf{U}_i^n - \frac{\Delta t}{A_i} \sum_{e \in \partial i} L_e \hat{\mathbf{F}}_{e,i} + \Delta t \mathbf{S}_i.$$

Surface physics: bed elevation, HLL flux, wetting/drying, semi-implicit Manning friction, rainfall and Horton infiltration.

Coupled processes: drainage capture, link routing, surcharge, outfalls, recharge, external inflow and outflow-only boundaries.

CFL-controlled adaptive timesteps keep the explicit update stable.

5. The workload is highly parallel—but structurally difficult



GPU opportunity

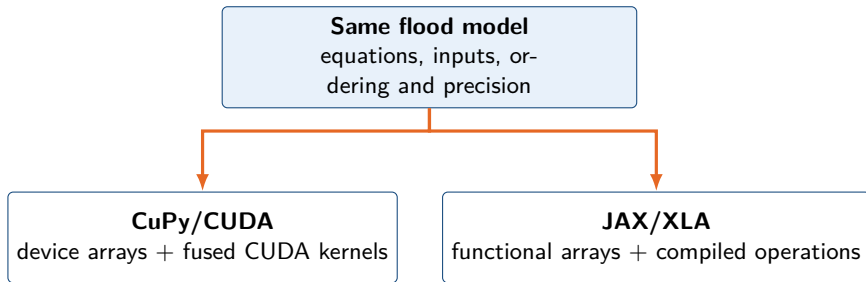
- ▶ Many edge, cell and link operations are independent.
- ▶ Major arrays can remain device-resident.

GPU challenge

- ▶ Irregular mesh connectivity
- ▶ Scatter/indexed reductions
- ▶ Wet-dry branching and safeguards
- ▶ Adaptive time integration
- ▶ Surface-network exchange ordering

This motivates two implementation strategies for one numerical model.

6. One flood model is expressed through two GPU backends



	CuPy/CUDA path	JAX/XLA path
Surface state	Mutable device arrays	Immutable named state tuple
Edge accumulation	Fused CUDA kernels	XLA scatter reductions
Time integration	Host-directed launches	Compiled <code>lax.while_loop</code>
Acceptance	Agreement of physical outputs and ledgers	

Different execution strategies do not imply different physics.

7. Both paths ran on one controlled GPU environment

	CuPy/CUDA	JAX/XLA
Recorded version	CuPy 14.1.1	JAX/jaxlib 0.10.2
Device	CUDA device 0	cuda:0
Surface precision	float32	float32
Execution style	Fused kernels	XLA JIT
GPU fallback	none	none

JAX execution discipline

Pure array functions are traced and lowered through XLA. Compilation is distinguished from steady execution, and accelerator work is synchronized before timing.

Source: completed-project evidence package and final report.

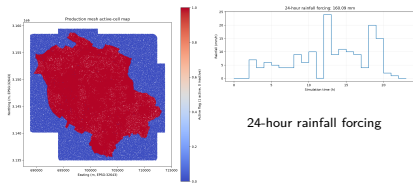
Recorded platform

NVIDIA RTX PRO 4000 Blackwell
24,467 MiB GPU memory
Driver 595.84
CuPy CUDA runtime 13.2
Python 3.12.14

PyTorch scope

PyTorch 2.13.0+cu130 was installed and CUDA availability was confirmed, but it was **not** a production or benchmark backend. No PyTorch timing claim is made.

8. A fair experiment holds the complete production case fixed



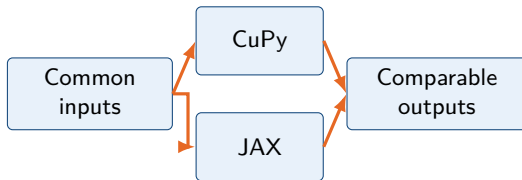
Production triangular domain

Active area	297.966 km ²
Triangles	981,880
Drainage nodes / links	118,768 / 139,798
Rainfall total	160.09 mm
Simulation / outputs	24 h / hourly

Source: completed-project evidence package and final report.

Common experimental controls

Both backends received the same hashed topology, terrain, Manning field, rainfall, infiltration, drainage, recharge, boundary and external-inflow data; the same state ordering, source sequence, float32 precision, CFL 0.85, 1 s maximum timestep, 5 m s⁻¹ velocity ceiling and output targets.

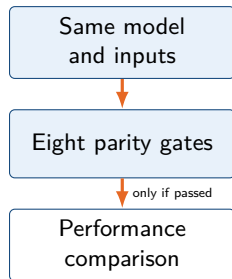


9. Correctness gates are passed before performance is compared

Predefined parity gate	Approved limit
Minimum hourly flood-extent CSI	≥ 0.95
Maximum hourly depth RMSE	≤ 0.05 m
Maximum hourly volume error	$\leq 2\%$
Finite/non-negative depth	minimum $\geq -10^{-7}$ m
Residual-ratio regression	increase ≤ 0.001
Maximum-depth relative error	$\leq 5\%$
Snapshot-time mismatch	≤ 1 s
Source-sink ledger difference	$\leq 2\%$

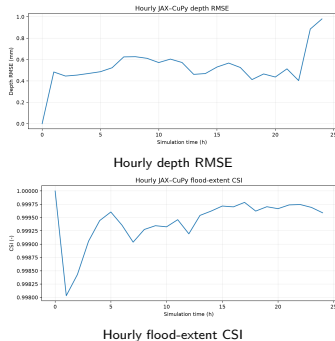
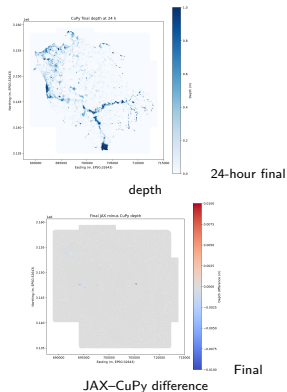
Acceptance philosophy

Limits were registered before interpreting the comparison. CuPy defines the numerical reference direction; it is not observed truth.



Speed is scientifically meaningful only after equivalence is established.

10. Production outputs passed all eight numerical parity gates



ALL 8 GATES PASSED

Maximum hourly depth RMSE
0.000979348 m

Minimum hourly CSI
0.998031

Difference between maximum final
depths
 $\approx 7.63 \times 10^{-6}$ m

The maximum hourly volume error was 0.033294%; the maximum snapshot-time mismatch was 0.318359 s. Every remaining registered gate also passed.

Source: completed-project evidence package and final report.

11. Independent repeats were numerically stable, not bitwise identical

CuPy repeat run

RMSE = 0.000416766 m

Minimum CSI = 0.998740

Maximum time offset = 0.354492 s

Step-count difference = 28

Registered criteria: PASSED

JAX repeat run

RMSE = 0.000406100 m

Minimum CSI = 0.998031

Maximum time offset = 0.337891 s

Step-count difference = 24

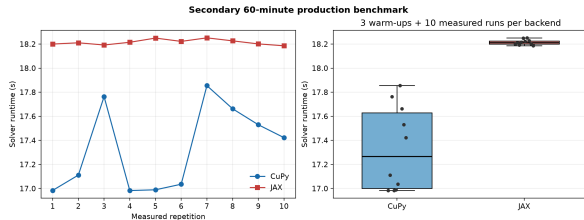
Registered criteria: PASSED

Correct interpretation

Both backends were numerically repeatable within the approved criteria, but neither archive was byte-for-byte identical. The evidence does not isolate the low-level cause of the small difference, so no unsupported explanation is assigned.

Source: completed-project evidence package and final report.

12. CuPy had the lower median runtime for the tested workload



Three warm-ups followed by ten synchronized measurements per backend

CuPy median
17.267 s

JAX median
18.213 s

Measured
difference
JAX
+5.476%

Completed secondary benchmark: identical 60-minute production workload; all twenty timed outputs passed correctness checks.

This is workload- and hardware-specific, not a universal CuPy-versus-JAX ranking.

Source: completed-project evidence package and final report.

13. The evidence supports numerical claims within clear boundaries

What the study demonstrates

- ▶ Complete city-scale surface–drainage GPU execution
- ▶ Cross-backend numerical parity
- ▶ Within-backend numerical repeatability
- ▶ A completed repeated 60-minute GPU comparison
- ▶ Reproducible inputs, tests, records and checksums

What it does not demonstrate

- ▶ No observations: no physical-validation claim
- ▶ One event: no event-to-event generalization
- ▶ Incomplete repeated 24-hour benchmark: no official ratio
- ▶ ANUGA controlled cases: numerical reference only
- ▶ No bitwise-determinism or production AI claim

Numerical agreement between implementations is not proof of real-world predictive accuracy.

14. Conclusion: equivalent GPU implementations were achieved

Direct answer to the research question

The same production flood model was implemented through CuPy/CUDA and JAX/XLA, and both paths satisfied the predefined numerical requirements on one NVIDIA GPU.

- ▶ All eight cross-backend parity gates passed.
- ▶ Both backends passed numerical-repeatability criteria.
- ▶ CuPy had the lower median runtime in the completed secondary benchmark for this workload.
- ▶ The result is a tested, reproducible and scientifically qualified GPU backend comparison.
- ▶ It is not evidence of observational flood accuracy or an operational AI forecasting system.

Verified physics → numerical parity → defensible performance

<https://github.com/arsanjeevk/gpu-urban-flood-cupy-jax>

Source: completed-project evidence package and final report.